

基于你提供的代码文件，以下是关于使用 **Meta Attributes**（元属性）、**SetByCaller** 的标准化流程，以及基于 Ability/Tag/GE 实现受伤反应的详细解析。

一、使用 **Meta Attributes (IncomingDamage)** 的必要性

在 GAS 系统中，直接修改核心属性（如 Health）通常是不推荐的做法，尤其是涉及伤害计算时。引入 Meta Attribute（如 IncomingDamage）作为中间缓冲层是必要的。

1. 概念定义

Meta Attribute 是一个临时的属性，它仅在 Gameplay Effect (GE) 执行期间存在数值，主要用作数据的“占位符”或“传输带”。它不会永久存储在 AttributeSet 中。

- 在代码中，IncomingDamage 被定义为 Meta Attribute。

2. 为什么必须使用它？

- 逻辑解耦与后处理**：如果直接修改 Health，你就失去了干预伤害计算的机会。使用 IncomingDamage，你可以先记录“原本应该造成的伤害量”，然后在应用到 Health 之前进行一系列复杂的计算（如减去护甲 Armor、计算暴击抗性、判断是否无敌等）。
- 原子性与清理**：Meta Attributes 的生命周期极短。在 PostGameplayEffectExecute 中，我们获取它的值，进行逻辑处理（扣血），然后立即将其重置为 0，确保下一次计算不受影响。

代码证据：

C++

```
// AuraAttributeSet.cpp
```

```
if(Data.EvaluatedData.Attribute == GetIncomingDamageAttribute())
```

```
{
```

```
    const float LocalIncomingDamage = GetIncomingDamage();
```

```
    SetIncomingDamage(0.f); // 立即消耗并清零
```

```
    if(LocalIncomingDamage > 0.f)
```

```
    {
```

```
        const float NewHealth = GetHealth() - LocalIncomingDamage; // 执行实际扣血逻辑
```

```
        SetHealth(FMath::Clamp(NewHealth, 0.f, GetMaxHealth()));
```

```
        // ...  
    }  
}
```

二、SetByCaller 修改幅度的标准流程

虽然提供的代码主要展示了接收端 (AttributeSet)，但结合 GAS 标准与 IncomingDamage 的存在，标准流程如下。这允许你在运行时动态传递伤害数值（例如武器造成的随机伤害）。

1. 发送端 (Gameplay Ability 中)

通常在造成伤害的 Gameplay Ability 中：

1. **创建 Spec**：使用 MakeOutgoingSpec 创建 GE 的实例。
2. **设置数值**：调用 AssignTagSetByCallerMagnitude。你需要一个 FGameplayTag（通常是 Data.Damage）作为 Key，并将计算出的具体伤害数值 (float) 作为 Value 传入 Spec。
3. **应用 GE**：调用 ApplyGameplayEffectSpecToTarget。

2. 接收端 (AttributeSet 中)

这是你提供的代码中展示的部分：

1. **GE 执行**：当 GE 到达目标 ASC 时，它会寻找与 IncomingDamage 绑定的 Modifier。
 2. **提取数值**：系统内部会根据 SetByCaller 传入的 Magnitude 修改 IncomingDamage 的值。
 3. **后处理执行**：触发 PostGameplayEffectExecute。此时 GetIncomingDamage() 获取到的就是你在 Ability 中通过 SetByCaller 传递进来的动态数值。
-

三、角色受击反应：Ability、Tag 与 GE 的协同实现

这是一个经典的“数据驱动逻辑”闭环：**数据变化 -> 触发逻辑 -> 改变表现**。

1. 第一阶段：AttributeSet 监测伤害

一切始于 PostGameplayEffectExecute。当 IncomingDamage 被应用并导致计算出新的生命值后，AttributeSet 会检查角色是否存活。

- **逻辑**：如果 !bFatal (没死)，代码会创建一个 FGameplayTagContainer 并添加标签 Effects_HitReact。
- **激活能力**：AttributeSet 直接调用
Props.TargetASC->TryActivateAbilitiesByTag(TagContainer)。这行代码尝试激活目标身上所有带有 Effects_HitReact 触发标签的 Gameplay Ability。

2. 第二阶段：Enemy 监听 Tag 变化

在 AAuraEnemy 中，并不直接通过 C++ 调用播放蒙太奇，而是通过监听 Gameplay Tag 的变化来驱动行为状态。

- **注册监听**：在 BeginPlay 中，敌人 ASC 注册了一个监听器，监听 Effects_HitReact 标签的添加或移除事件。

C++

```
AbilitySystemComponent->RegisterGameplayTagEvent(
    FAuraGameplayTags::Get().Effects_HitReact,
    EGameplayTagEventType::NewOrRemoved
).AddUObject(this, &AAuraEnemy::HitReactTagChanged);
```

3. 第三阶段：执行表现 (HitReactTagChanged)

当上述 Ability 被激活（通常该 Ability 会在该期间给自己赋予 Effects_HitReact 标签），回调函数 HitReactTagChanged 被触发。

- **状态改变**：函数接收 NewCount。如果 NewCount > 0，说明角色正在受击（Ability 正在运行）。
- **移动控制**：通过修改 GetCharacterMovement()->MaxWalkSpeed，将速度设为 0，实现“受击硬直”效果；当 Ability 结束（Tag 移除），速度恢复 BaseWalkSpeed。

总结流程图

1. **GE (SetByCaller)** 应用 IncomingDamage。
2. **AttributeSet** 处理 IncomingDamage -> 扣除 Health。
3. **AttributeSet** 发现角色未死 -> TryActivateAbilitiesByTag(HitReact)。
4. **HitReact Ability** (未展示代码，但在蓝图中) 激活 -> 给自己添加 HitReact Tag。
5. **AAuraEnemy** 检测到 HitReact Tag 增加 -> HitReactTagChanged -> **停止移**

动。

6. **HitReact Ability** 播放蒙太奇结束 -> 移除 HitReact Tag。
7. **AAuraEnemy** 检测到 HitReact Tag 减少 -> 恢复移动。